MYSQL
PART 1

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to MySQL and RDBMS

1.1 What is a Database ?

- A database is an organized collection of data that can be easily stored, managed and retrieved.
- It helps in storing large amount of data in a structured format.
- Example: Bank data, employee records, product information etc.

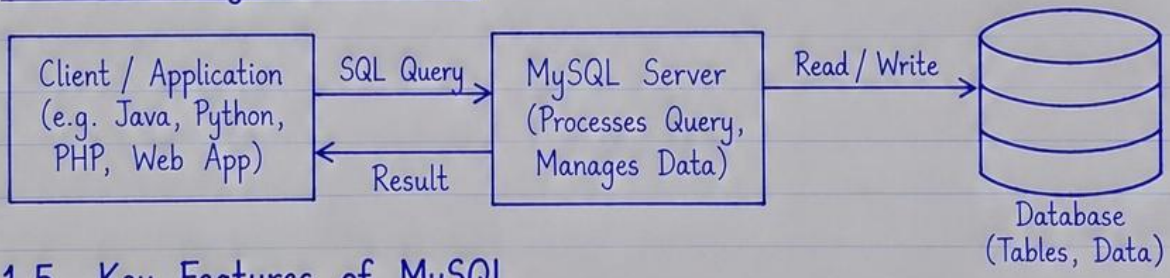
1.2 What is RDBMS ?

- RDBMS (Relational Database Management System) stores data in tables (rows and columns).
- Data in different tables can be related using keys.
- Examples: MySQL, PostgreSQL, Oracle, SQL Server.

1.3 What is MySQL ?

- MySQL is an open-source RDBMS used to store and manage data.
- It uses SQL (Structured Query Language) to perform operations.
- It is fast, reliable, easy to use and widely used in web applications.
- It supports large databases and is maintained by Oracle.

1.4 How MySQL Works ?



1.5 Key Features of MySQL

- Open source and free to use
- High performance and reliable
- Supports large databases
- Easy to learn and use
- Supports multiple storage engines
- Cross-platform (Windows, Linux, Mac)
- Widely used in web applications
- Good community support

1.6 RDBMS vs MySQL

Feature	RDBMS	MySQL
Full Form	Relational Database Management System	MySQL (an RDBMS)
Type	Category of database system	Specific database software
Example	MySQL, PostgreSQL, Oracle, SQL Server	Only MySQL
License	Can be open source or commercial	Open source (Community Edition)
Language	Uses SQL	Uses SQL
Data Storage	Stores data in tables	Stores data in tables
Usage	General term for relational DB systems	Widely used for web applications

Rahul Yadav

2. MySQL Installation and Setup

2.1 Ways to Install MySQL

- Windows – MySQL Installer (GUI based)
- Linux – Package manager (apt, yum)
- Mac – Homebrew
- Docker – Using MySQL Docker image

2.2 System Requirements

- OS: Windows 10/11, Linux (Ubuntu/CentOS), macOS
- RAM: Minimum 2 GB (4 GB recommended)
- Disk Space: At least 2 GB
- MySQL Version: 8.0 (recommended)

2.3 Installation Steps (Windows – MySQL Installer)

1. Download MySQL Installer from official website (https://dev.mysql.com).
2. Run the installer and choose "Developer Default".
3. Follow the installation steps.
4. Set the root password.
5. Complete the installation and start MySQL Server.

2.4 Connecting to MySQL

- Using MySQL Command Line Client:

```
mysql -u root -p
Enter password: *******
```

- Using MySQL Workbench:
  - Open MySQL Workbench.
  - Create a new connection.
  - Enter host (localhost), port (3306), username (root) and password.

2.5 Important MySQL Directories (Windows Example)

Directory	Description
C:\Program Files\MySQL\MySQL Server 8.0	Main installation directory
...\bin	Contains mysql.exe and other tools
...\data	Stores database files
...\my.ini	Configuration file
...\logs	Stores server logs

2.6 Basic MySQL Commands After Installation

```
# Login to MySQL
mysql -u root -p

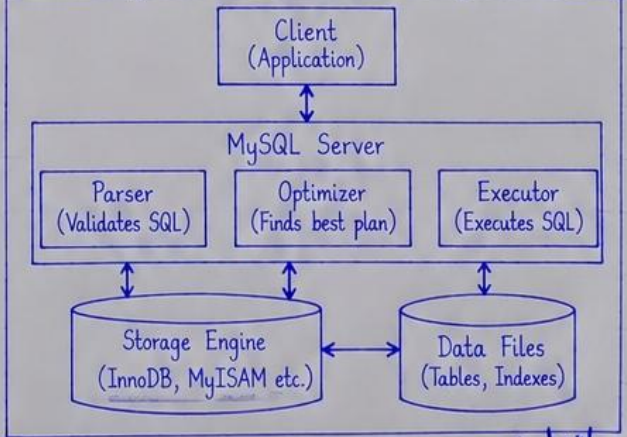
# Check MySQL version
SELECT VERSION();

# Show current user
SELECT USER();

# Show available databases
SHOW DATABASES;

# Exit from MySQL
EXIT;
```

2.7 MySQL Architecture (Simple View)



Rahul Yadav

3. Databases and Tables

3.1 What is a Database ?

- A database is a collection of related tables.
- It helps in organizing and storing data efficiently.
- In MySQL, we can create multiple databases.
- Example: BankDB, EmployeeDB, ECommerceDB etc.

3.2 What is a Table ?

- A table is a collection of data stored in rows and columns.
- Each column represents a field (attribute) and each row represents a record.
- Every table must have a unique name within a database.
- Example: customers, employees, products etc.

3.3 Create, Use and Show Databases

```
-- Create a new database
CREATE DATABASE BankDB;
-- Show all databases
SHOW DATABASES;
-- Select a database to use
USE BankDB;
-- Check current database
SELECT DATABASE();
-- Delete a database
DROP DATABASE BankDB;
```

Output [SHOW DATABASES;]

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| BankDB             |
+--------------------+
```

3.4 Create a Table

```
CREATE TABLE customers (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Explanation:

- id → Integer, Primary Key
- name → String (max 100 chars)
- email → Unique value
- age → Integer
- created_at → Stores current date and time automatically

3.5 Data Types in MySQL (Commonly Used)

Data Type	Description	Example
INT	Integer number	id INT
VARCHAR(n)	Variable length string	name VARCHAR(100)
TEXT	Large text data	description TEXT
DATE	Stores date (YYYY-MM-DD)	dob DATE
DATETIME	Stores date and time	created_at DATETIME
BOOLEAN	True / False (0 or 1)	is_active BOOLEAN
DECIMAL(p, s)	Fixed point number (p = total digits, s = decimals)	price DECIMAL(10, 2)

3.6 View Table Structure and Details

```
-- Show table structure
DESCRIBE customers;

-- Another way
SHOW COLUMNS FROM customers;

-- Show create table statement
SHOW CREATE TABLE customers;
```

Output (DESCRIBE customers;)

```
+------------+-------------+------+-----+-------------------+-------+
| Field      | Type        | Null | Key | Default           | Extra |
+------------+-------------+------+-----+-------------------+-------+
| id         | int         | NO   | PRI | NULL              |       |
| name       | varchar(100)| NO   |     | NULL              |       |
| email      | varchar(100)| YES  | UNI | NULL              |       |
| age        | int         | YES  |     | NULL              |       |
| created_at | timestamp   | YES  |     | CURRENT_TIMESTAMP |       |
+------------+-------------+------+-----+-------------------+-------+
```

Rahul Yadav

4. Data Types and Constraints

4.1 MySQL Data Types

- MySQL provides different data types to store different kind of data.
- Choosing the correct data type is important for performance and storage.
- Major categories: Numeric, String, Date & Time, Boolean, Binary, JSON etc.

4.2 Numeric Data Types

Type	Description	Example
INT	Integer number	10, -5
BIGINT	Large integer	1234567890
DECIMAL(p,s)	Fixed point number	DECIMAL(10,2)
FLOAT	Single precision	3.14
DOUBLE	Double precision	3.141592
TINYINT	Small integer	0 - 255

4.3 String Data Types

Type	Description	Example
CHAR(n)	Fixed length string	CHAR(10)
VARCHAR(n)	Variable length string	VARCHAR(100)
TEXT	Large text (up to 64KB)	TEXT
MEDIUMTEXT	Medium text (up to 16MB)	MEDIUMTEXT
LONGTEXT	Very large text (up to 4GB)	LONGTEXT
ENUM	Predefined list of values	ENUM('M','F')
SET	Multiple predefined values	SET('A','B','C')

4.4 Date and Time Data Types

Type	Description	Example
DATE	Stores date (YYYY-MM-DD)	2024-10-15
DATETIME	Stores date and time	2024-10-15 14:30:00
TIMESTAMP	Auto-updated timestamp	CURRENT_TIMESTAMP
TIME	Stores time	14:30:00
YEAR	Stores year	2024

4.5 Other Important Data Types

Type	Description	Example
BOOLEAN	True / False (0 or 1)	1, 0
BINARY(n)	Fixed length binary data	BINARY(16)
VARBINARY(n)	Variable length binary data	VARBINARY(100)
JSON	Stores JSON data	JSON
BLOB	Binary large object	BLOB
UUID	Stores UUID value	UUID()

4.6 Constraints in MySQL

- Constraints are used to enforce rules on data in a table.
- They help maintain data integrity and consistency.
- Constraints can be defined at column level or table level.

Constraint	Description	Example
PRIMARY KEY	Uniquely identifies each record	id INT PRIMARY KEY
FOREIGN KEY	Establishes relationship between tables	FOREIGN KEY (dept_id) REFERENCES dept(id)
UNIQUE	Ensures all values are unique	email VARCHAR(100) UNIQUE
NOT NULL	Value cannot be NULL	name VARCHAR(100) NOT NULL
DEFAULT	Sets a default value	status VARCHAR(20) DEFAULT 'Active'
CHECK	Validates data based on condition	age INT CHECK (age >= 18)
AUTO_INCREMENT	Automatically increases value	id INT AUTO_INCREMENT
INDEX	Improves query performance	INDEX (name)

4.7 Create Table with Different Data Types and Constraints

```
CREATE TABLE employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age >= 18),
    salary DECIMAL(10, 2) DEFAULT 0.00,
    joining_date DATE,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(id)
);
```

4.8 Output

Table: employees

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(100)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
age	int	NO		NULL	
salary	decimal(10,2)	YES		0.00	
joining_date	date	YES		NULL	
dept_id	int	YES	MUL	NULL	

Rahul Yadav

5. SQL Queries - Basic Operations

5.1 What is SQL ?

- SQL (Structured Query Language) is used to communicate with and manipulate data in a database.
- It is used to perform operations like retrieve, insert, update, delete data.
- SQL works with relational databases like MySQL, Oracle, PostgreSQL etc.

5.2 Types of SQL Commands

Type	Description	Examples
DDL (Data Definition Language)	Used to define and manage database objects	CREATE, ALTER, DROP, TRUNCATE
DML (Data Manipulation Language)	Used to manipulate data in tables	SELECT, INSERT, UPDATE, DELETE
DQL (Data Query Language)	Used to query and retrieve data	SELECT
DCL (Data Control Language)	Used to control access to data	GRANT, REVOKE
TCL (Transaction Control Language)	Used to manage transactions	COMMIT, ROLLBACK, SAVEPOINT

5.3 CREATE Database and Table

```
-- Create a database
CREATE DATABASE BankDB;
-- Use the database
USE BankDB;
-- Create a table
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT,
    salary DECIMAL(10,2),
    department VARCHAR(50)
);
```

5.4 INSERT Data into Table

```
-- Insert a single record
INSERT INTO employees (id, name, age, salary, department)
VALUES (1, 'Rahul', 28, 75000.00, 'IT');

-- Insert multiple records
INSERT INTO employees (id, name, age, salary, department)
VALUES
    (2, 'Preeti', 26, 65000.00, 'HR'),
    (3, 'Amit', 30, 80000.00, 'Finance'),
    (4, 'Neha', 25, 60000.00, 'Operations');
```

5.5 SELECT Query - Retrieve Data

```
-- Select all columns
SELECT * FROM employees;
-- Select specific columns
SELECT id, name, department FROM employees;
-- Select with condition
SELECT * FROM employees WHERE age > 25;
-- Select with multiple conditions
SELECT * FROM employees
WHERE department = 'IT' AND salary > 70000;
```

Output:

id	name	age	salary	department
1	Rahul	28	75000.00	IT
2	Preeti	26	65000.00	HR
3	Amit	30	80000.00	Finance
4	Neha	25	60000.00	Operations

5.6 UPDATE and DELETE Query

```
-- Update data
UPDATE employees
SET salary = 85000.00
WHERE id = 1;

-- Delete data
DELETE FROM employees
WHERE id = 4;
```

5.7 TRUNCATE vs DELETE vs DROP

Command	Description	Effect
DELETE	Deletes specific rows or all rows (with condition)	Can be rolled back (Slower)
TRUNCATE	Removes all rows from table	Cannot be rolled back (Faster)
DROP	Removes the entire table structure	Cannot be rolled back (Permanent)

Rahul Yadav

6. WHERE Clause and Operators

6.1 What is WHERE Clause ?

- WHERE clause is used to filter records based on a condition.
- It is used with SELECT, UPDATE, DELETE statements.
- It helps to retrieve only the required data from a table.
- We can use comparison operators, logical operators, pattern matching etc. in WHERE clause.

6.2 Comparison Operators

Operator	Description	Example	Meaning
=	Equal to	age = 25	Age is equal to 25
!= or <>	Not equal to	age != 25	Age is not equal to 25
>	Greater than	salary > 50000	Salary greater than 50000
<	Less than	age < 30	Age less than 30
>=	Greater than or equal to	age >= 25	Age greater than or equal to 25
<=	Less than or equal to	salary <= 75000	Salary less than or equal to 75000

6.3 Logical Operators

Operator	Description	Example
AND	Both conditions true	age > 25 AND salary > 50000
OR	Any one condition true	age < 25 OR department = 'HR'
NOT	Negates the condition	NOT department = 'IT'

6.4 LIKE Operator (Pattern Matching)

- LIKE operator is used to search for a specified pattern in a column.
- % → Represents zero or more characters
- _ → Represents a single character

Example	Meaning
name LIKE 'R%'	Name starts with R
name LIKE '%a'	Name ends with a
name LIKE '%ah%'	Name contains 'ah'
name LIKE 'R_hul'	Second character can be any one

6.5 IN Operator

- IN operator is used to specify multiple values in a WHERE clause.
- It is a shorthand for multiple OR conditions.

```
SELECT * FROM employees
WHERE department IN ('IT', 'HR', 'Finance);
```

6.6 BETWEEN Operator

- BETWEEN operator is used to filter values within a given range (inclusive).

```
SELECT * FROM employees
WHERE age BETWEEN 25 AND 30;
```

6.7 IS NULL and IS NOT NULL

- IS NULL is used to check for NULL values.
- IS NOT NULL is used to check for non-NULL values.

```
SELECT * FROM employees WHERE email IS NULL;
SELECT * FROM employees WHERE email IS NOT NULL;
```

6.8 ORDER BY Clause

- ORDER BY is used to sort the result set.
- ASC → Ascending order (default)
- DESC → Descending order

```
SELECT * FROM employees
WHERE department = 'IT'
ORDER BY salary DESC;
```

6.9 LIMIT Clause (Fetch Limited Records)

- LIMIT clause is used to return a limited number of records.
- It is useful when we only need top N records.

```
SELECT * FROM employees
ORDER BY salary DESC
LIMIT 5;
```

6.10 Combined Example

- Using multiple conditions with WHERE, LIKE, BETWEEN, ORDER BY and LIMIT.

```
SELECT id, name, age, salary, department
FROM employees
WHERE department LIKE 'F%' AND age BETWEEN 25 AND 35
ORDER BY salary DESC
LIMIT 10;
```

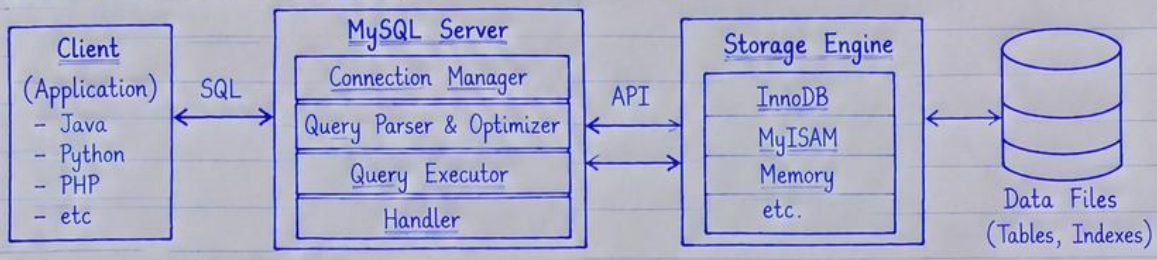
Rahul Yadav

6. MySQL Architecture

6.1 Overview

- MySQL follows a client-server architecture.
- It has a layered structure that separates the client, server and storage engine.
- It supports multiple storage engines (InnoDB, MyISAM etc).

6.2 Architecture Diagram



6.3 Layers in MySQL

Layer	Description
Client Layer	Application connects to MySQL using a client library or tool.
Connection Manager	Manages client connections and authentication.
Query Parser	Checks syntax and builds an internal representation (parse tree).
Optimizer	Chooses the best execution plan for the query.
Query Executor	Executes the plan and gets data from storage engine.
Storage Engine	Handles data storage, indexing, transactions etc.
Data Files	Actual data is stored in files on disk.

6.4 Storage Engines

- InnoDB – Supports transactions, row-level locking, foreign keys (default).
- MyISAM – Fast, simple, no transactions.
- Memory – Data stored in RAM (temporary).
- Archive – For large, write-once tables.
- CSV – Stores data in CSV format.

6.5 InnoDB vs MyISAM

Feature	InnoDB	MyISAM
Transactions	Yes	No
Row Level Locking	Yes	No
Foreign Keys	Yes	No
Full Text Search	Yes	Yes
Speed (Read)	Good	Very Fast
Use Case	OLTP (high update)	Read heavy

6.6 Flow of a SELECT Query

1. Client sends SQL query.
2. Connection manager checks authentication.
3. Parser checks syntax.
4. Optimizer creates execution plan.
5. Executor calls storage engine.
6. Storage engine reads data from files.
7. Result is sent back to client.

6.7 Key Files and Directories

File / Directory	Purpose
my.cnf / my.ini	Configuration file
data/	Database files
ibdata1	InnoDB system tablespace
.frm	Table structure (older versions)
.ibd	InnoDB table data (separate)
logs/	Error and general logs

6.8 Important Points

- Use InnoDB for production systems.
- Configure buffer pool size for better performance.
- Keep proper indexes for faster queries.
- Monitor slow queries using slow_query_log.

6.9 Real World Use Case

- E-commerce website (product data, orders, users).
- Banking system (transactions, account data).
- Social media (posts, comments, user info).
- Used in almost all web applications.

Rahul Yadav

7. Aggregate Functions and GROUP BY

7.1 What are Aggregate Functions ?

- Aggregate functions are used to perform calculations on a set of rows and return a single value.
- They are commonly used with GROUP BY clause.
- They ignore NULL values (except COUNT(*)).
- Common aggregate functions: COUNT(), SUM(), AVG(), MIN(), MAX().

Common Aggregate Functions

Function	Description
COUNT()	Returns total number of rows
COUNT(column)	Counts non-NULL values
SUM()	Returns sum of numeric values
AVG()	Returns average of numeric values
MIN()	Returns minimum value
MAX()	Returns maximum value

7.2 Aggregate Functions Examples

```
SELECT COUNT(*) AS total_employees,
       COUNT(department) AS dept_count,
       SUM(salary) AS total_salary,
       AVG(salary) AS avg_salary,
       MIN(salary) AS min_salary,
       MAX(salary) AS max_salary
  FROM employees;
```

Output

total_employees	dept_count	total_salary	avg_salary	min_salary	max_salary
4	4	275000.00	68750.00	60000.00	80000.00

7.3 GROUP BY Clause

- GROUP BY is used to group rows that have the same values in specified column(s).
- It is often used with aggregate functions.
- It returns one row per group.

Example: Total employees and average salary per department

```
SELECT department,
       COUNT(*) AS emp_count,
       AVG(salary) AS avg_salary
  FROM employees
 GROUP BY department;
```

department	emp_count	avg_salary
Finance	1	80000.00
HR	1	65000.00
IT	1	75000.00
Operations	1	60000.00

7.4 GROUP BY with Multiple Columns

```
SELECT department, age,
       COUNT(*) AS emp_count
  FROM employees
 GROUP BY department, age;
```

Output

department	age	emp_count
Finance	30	1
HR	26	1
IT	28	1
Operations	25	1

7.5 HAVING Clause

- HAVING is used to filter groups after applying aggregate functions.
- WHERE cannot be used with aggregate functions, so we use HAVING.

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 0;
```

Example: Departments with average salary > 65000

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 65000;
```

Output

department	avg_salary
Finance	80000.00
IT	75000.00

7.6 DISTINCT Keyword

- DISTINCT is used to return only unique values.
- It removes duplicate rows from the result set.
- We can use DISTINCT with single or multiple columns.

```
SELECT DISTINCT department FROM employees;
```

Example: Get unique departments

```
SELECT DISTINCT department FROM employees;
```

Output

department
Finance
HR
IT
Operations

Rahul Yadav

8. JOINs in MySQL

8.1 What is JOIN ?

- JOIN is used to combine rows from two or more tables based on a related column between them.
- It helps to fetch related data stored in multiple tables.
- Types of JOIN: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN, SELF JOIN.

Sample Tables

employees

id	name	dept_id
1	Rahul	1
2	Preeti	2
3	Amit	1
4	Neha	3

departments

id	dept_name
1	IT
2	HR
3	Finance
4	Operations

8.2 INNER JOIN

- Returns only matching records from both tables.
- Most commonly used join.

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.id;
```

Output

name	dept_name
Rahul	IT
Preeti	HR
Amit	IT
Neha	Finance

8.3 LEFT JOIN

- Returns all records from left table and matching records from right table.
- If no match, right table columns will be NULL.

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.id;
```

Output

name	dept_name
Rahul	IT
Preeti	HR
Amit	IT
Neha	Finance

8.4 RIGHT JOIN

- Returns all records from right table and matching records from left table.
- If no match, left table columns will be NULL.

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.id;
```

Output

name	dept_name
Rahul	IT
Preeti	HR
Amit	IT
Neha	Finance
NULL	Operations

8.5 FULL JOIN (Using UNION)

- MySQL does not have FULL JOIN directly.
- We can achieve it using LEFT JOIN and RIGHT JOIN with UNION.

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.id
UNION
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.id;
```

Output

name	dept_name
Rahul	IT
Preeti	HR
Amit	IT
Neha	Finance
NULL	Operations

8.6 CROSS JOIN

- Returns the Cartesian product of both tables.
- Each row from first table is combined with each row from second table.

```
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

Output

name	dept_name
Rahul	IT
Rahul	HR
Rahul	Finance
...	...

8.7 SELF JOIN

- A table is joined with itself.
- Useful when we need to compare rows within the same table.

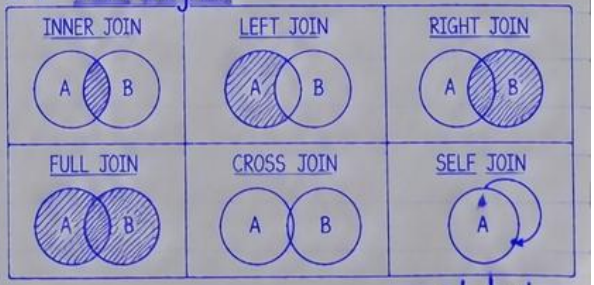
```
SELECT e1.name AS employee,
    e2.name AS manager
FROM employees e1
LEFT JOIN employees e2
ON e1.manager_id = e2.id;
```

employee	manager
Rahul	NULL
Preeti	Rahul
Amit	Rahul
Neha	Preeti

8.8 Types of JOIN - Summary

Join Type	Returns	Use Case
INNER JOIN	Matching rows only	Commonly used
LEFT JOIN	All left + matched right	Keep all left table data
RIGHT JOIN	All right + matched left	Keep all right table data
FULL JOIN	All records from both	Complete data (use UNION)
CROSS JOIN	Cartesian product	All possible combinations
SELF JOIN	Table with itself	Hierarchical or comparative data

8.9 JOIN Diagram



Rahul Yadav

9. Subqueries in MySQL

9.1 What is Subquery ?

- A subquery is a query within another query.
- The inner query (subquery) is executed first, and its result is used by the outer query.
- Subqueries can be used in SELECT, WHERE, FROM and HAVING clause.
- Subquery returns a single value, multiple values or a table depending on the use case.

Types of Subqueries

Type	Description
Single Row Subquery	Returns only one value (one row)
Multiple Row Subquery	Returns multiple values (multiple rows)
Multiple Column Subquery	Returns multiple columns
Correlated Subquery	Depends on outer query for each row
Non-Correlated Subquery	Independent of outer query
Subquery in SELECT	Used in SELECT clause
Subquery in FROM	Used in FROM clause (derived table)
Subquery in WHERE	Used in WHERE clause
Subquery in HAVING	Used in HAVING clause

9.2 Single Row Subquery

- Returns only one row and one column.
- Can use operators like =, <, >, <=, >=, <>.

```
SELECT name, salary
FROM employees
WHERE salary > (
      SELECT AVG(salary)
      FROM employees
);
```

Output

name	salary
Rahul	75000.00
Amit	80000.00

9.3 Multiple Row Subquery

- Returns multiple rows.
- Used with IN, ANY, ALL operators.

```
SELECT name, department
FROM employees
WHERE department IN (
   SELECT dept_name
   FROM departments
   WHERE location = 'Bangalore'
);
```

Output

name	department
Rahul	IT
Neha	Finance

9.4 Multiple Column Subquery

- Returns multiple columns.
- Used with IN operator.

```
SELECT name, department, salary
FROM employees
WHERE (department, salary) IN (
    SELECT dept_name, MAX(salary)
    FROM employees
    GROUP BY dept_name
);
```

Output

name	department	salary
Amit	Finance	80000.00
Rahul	IT	75000.00

9.5 Correlated Subquery

- Depends on outer query.
- Executed for each row of outer query.

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary > (
     SELECT AVG(e2.salary)
     FROM employees e2
     WHERE e2.department =
           e1.department
);
```

Output

name	salary
Rahul	75000.00
Amit	80000.00

9.6 Subquery in SELECT Clause

- Used to display computed value for each row.

```
SELECT name,
      salary,
    (SELECT AVG(salary)
    FROM employees) AS avg_salary
FROM employees;
```

Output

name	salary	avg_salary
Rahul	75000.00	68750.00
Preeti	65000.00	68750.00

9.7 Subquery in FROM Clause (Derived Table)

- Subquery is used as a temporary table.
- Useful for complex queries.

```
SELECT dept_name, avg_sal
FROM (
   SELECT department AS dept_name,
          AVG(salary) AS avg_sal
   FROM employees
   GROUP BY department
) AS dept_avg;
```

Output

dept_name	avg_sal
Finance	80000.00
HR	65000.00
IT	75000.00
Operations	60000.00

9.8 Subquery in HAVING Clause

- Used to filter groups based on subquery result.

```
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING AVG(salary) > (
     SELECT AVG(salary) FROM employees
);
```

Output

department	avg_sal
Finance	80000.00
IT	75000.00

9.9 EXISTS vs IN

- EXISTS checks if subquery returns any row.
- IN checks for matching values.

Feature	EXISTS	IN
Returns	TRUE/FALSE	Matching values
Performance	Faster (for large data)	Slower (for large data)
Use case	Check existence	Compare values
Example	WHERE EXISTS (...)	WHERE id IN (...)

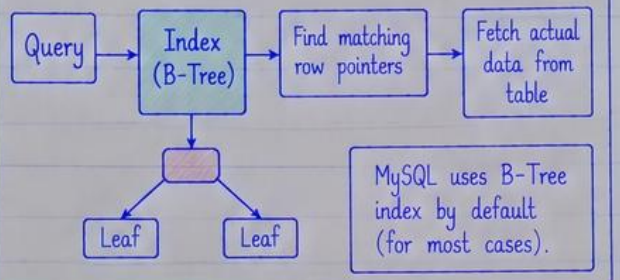
Rahul Yadav

10. Indexes in MySQL

10.1 What is an Index ?

- Index is a special data structure that improves the speed of data retrieval operations on a table.
- It works like an index in a book (helps to quickly find data without scanning entire table).
- Indexes are mainly used on columns which are frequently used in WHERE, JOIN, ORDER BY and GROUP BY clauses.

How Index Works ?



10.2 Types of Indexes

Index Type	Description	Use Case
Primary Index	Created automatically on PRIMARY KEY.	Uniquely identifies each row.
Unique Index	Ensures all values are unique.	Prevent duplicate values.
Normal Index	Speeds up data retrieval.	Used with WHERE, JOIN etc.
Composite Index	Index on multiple columns.	Used when queries have multiple conditions.
Fulltext Index	Used for full text search (on text columns).	Search in large text data.

10.3 Index vs No Index

Feature	With Index	Without Index
Data Retrieval Speed	Faster	Slower (full table scan)
Preformance	Better for large data	Poor for large data
Storage Space	Requires extra space	Less storage
Insert/Update/Delete	Slightly slower (need to update index)	Faster
Use Case	Frequently searched columns	Small tables or rarely searched data

10.4 Create and Drop Index

```
-- Create index on single column
CREATE INDEX idx_salary ON employees (salary);

-- Create unique index
CREATE UNIQUE INDEX idx_email ON employees (email);

-- Create composite index
CREATE INDEX idx_dept_salary ON employees (department, salary);

-- Drop index
DROP INDEX idx_salary ON employees;
```

10.5 View Indexes

```
-- Show indexes of a table
SHOW INDEXES FROM employees;
-- Using INFORMATION_SCHEMA
SELECT index_name, column_name, non_unique
FROM information_schema.statistics
WHERE table_name = 'employees';
```

10.6 Example Table

id	name	department	salary	email
1	Rahul	IT	75000	rahul@example.com
2	Preeti	HR	65000	preeti@example.com
3	Amit	Finance	80000	amit@example.com
4	Neha	Operations	60000	neha@example.com

10.7 Query Example with Index

```
-- Query using index (faster)
SELECT * FROM employees
WHERE department = 'IT' AND salary > 70000;

-- Without index (full table scan)
SELECT * FROM employees
WHERE salary > 70000;
```

10.8 Composite Index Example

- If we frequently search by department and salary together, we can create a composite index.
- The order of columns in composite index matters.

```
CREATE INDEX idx_dept_salary ON employees (department, salary);
SELECT * FROM employees
WHERE department = 'IT' AND salary > 70000;
```

10.9 EXPLAIN Query (Check Index Usage)

- We can use EXPLAIN to check if MySQL is using index or not.
- Look at the key column in output.

```
EXPLAIN SELECT * FROM employees
WHERE department = 'IT' AND salary > 70000;
```

Sample Output

id	select_type	table	type	key	rows	Extra
1	SIMPLE	employees	ref	idx_dept_salary	10	Using index condition

10.10 Best Practices

- Create indexes on columns used in WHERE, JOIN, ORDER BY and GROUP BY.
- Use composite index for multiple column queries.
- Avoid too many indexes (increases storage and slows down write operations).
- Use appropriate column order in composite index.
- Regularly monitor index usage.
- Use EXPLAIN to analyze queries.
- Remove unused indexes.

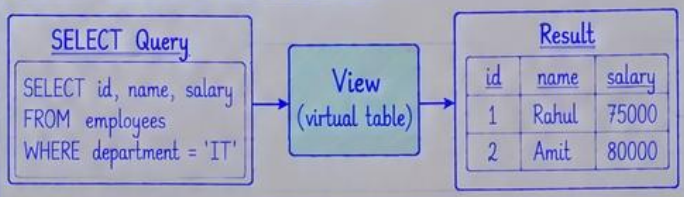
Rahul Yadav

11. Views in MySQL

11.1 What is a View ?

- A view is a virtual table based on the result of a SELECT query.
- It does not store data physically (just the query is stored).
- Views make complex queries simpler.
- They provide an extra layer of security by hiding sensitive data.
- When we query a view, MySQL executes the underlying SELECT query and returns the result.

11.2 How Views Work ?



The view stores only the query, not the actual data. Whenever we query the view, MySQL runs the query and fetches latest data from the underlying tables.

11.3 Create a View

```
CREATE VIEW it_employees AS
SELECT id, name, department, salary
FROM employees
WHERE department = 'IT';
```

11.4 Query a View

```
SELECT * FROM it_employees;
```

Output

id	name	department	salary
1	Rahul	IT	75000
2	Amit	IT	80000

11.5 View Definition

```
SHOW CREATE VIEW it_employees;
```

Output (simplified)

View	Create View
it_employees	CREATE ALGORITHM=UNDEFINED DEFINER=`root`@`localhost` SQL SECURITY DEFINER VIEW `it_employees` AS SELECT id, name, department, salary FROM employees WHERE department = 'IT'

11.6 Update a View

```
CREATE OR REPLACE VIEW it_employees AS
SELECT id, name, department, salary, email
FROM employees
WHERE department = 'IT';
```

11.7 Drop a View

```
DROP VIEW it_employees;
```

11.8 View with JOIN

```
CREATE VIEW emp_dept_view AS
SELECT e.id, e.name, e.salary, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

Output

id	name	salary	dept_name
1	Rahul	75000	IT
2	Preeti	65000	HR

11.9 View with Aggregate Function

```
CREATE VIEW dept_salary_summary AS
SELECT department, COUNT(*) AS emp_count,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Output

department	emp_count	avg_salary
Finance	4	80000
HR	3	65000
IT	5	75000
Operations	2	60000

11.10 Updatable vs Non-Updatable Views

Feature	Updatable View	Non-Updatable View
Underlying Query	Simple SELECT	Complex (JOIN, GROUP BY, DISTINCT, etc.)
Supports INSERT	Yes	No
Supports UPDATE	Yes	No
Supports DELETE	Yes	No
Use Case	Simple filtering	Reporting, aggregation, complex logic

11.11 Advantages of Views

- Simplifies complex queries.
- Hides sensitive data (security).
- Provides a consistent interface.
- Can be used for reporting.
- Helps in data abstraction.
- Useful in large applications.

11.12 Limitations of Views

- Views do not store data (slower compared to table).
- Not always updatable (complex views).
- Performance depends on underlying query.
- Cannot have indexes on views directly.
- Some operations (INSERT/UPDATE/DELETE) may not be supported.

11.13 Real World Use Cases

- Show limited columns to certain users.
- Create a simplified view for reporting dashboards.
- Hide sensitive data like salary, email, etc.
- Combine data from multiple tables.
- Provide consistent data to applications.

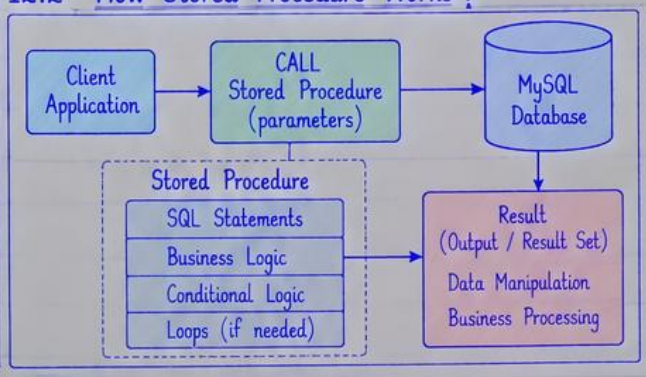
Rahul Yadav

12. Stored Procedures in MySQL

12.1 What is a Stored Procedure ?

- A stored procedure is a set of SQL statements that are stored in the database and can be executed multiple times.
- It helps to reuse code, reduce network traffic and improve performance.
- Stored procedures can accept input parameters, return output using OUT parameters and contain control flow logic (IF, LOOP, etc).
- They are useful for complex business logic and data manipulation.

12.2 How Stored Procedure Works ?



12.3 Create a Simple Stored Procedure

```
DELIMITER //
CREATE PROCEDURE get_all_employees()
BEGIN
    SELECT id, name, department, salary
    FROM employees;
END //
DELIMITER ;
```

12.4 Call a Stored Procedure

```
CALL get_all_employees();
```

Output

id	name	department	salary
1	Rahul	IT	75000
2	Preeti	HR	65000
3	Amit	Finance	80000

12.5 Stored Procedure with Input Parameter

```
DELIMITER //
CREATE PROCEDURE get_employee_by_dept(IN dept_name VARCHAR(50))
BEGIN
    SELECT id, name, department, salary
    FROM employees
    WHERE department = dept_name;
END //
DELIMITER ;
```

Call

```
CALL get_employee_by_dept('IT');
```

Output

id	name	department	salary
1	Rahul	IT	75000
3	Neha	IT	70000

12.6 Stored Procedure with OUT Parameter

```
DELIMITER //
CREATE PROCEDURE get_employee_count(
    OUT emp_count INT
)
BEGIN
    SELECT COUNT(*) INTO emp_count FROM employees;
END //
DELIMITER ;
```

Call

```
CALL get_employee_count(@total);
```

Check Output

```
SELECT @total;
```

Output

@total
4

12.7 Stored Procedure with IN and OUT Parameters

```
DELIMITER //
CREATE PROCEDURE get_avg_salary(
    IN dept_name VARCHAR(50),
    OUT avg_sal DECIMAL(10,2)
)
BEGIN
    SELECT AVG(salary) INTO avg_sal
    FROM employees
    WHERE department = dept_name;
END //
DELIMITER ;
```

Call

```
CALL get_avg_salary('IT', @avg);
SELECT @avg;
```

Output

@avg
72500.00

12.8 Control Flow in Stored Procedure

```
DELIMITER //
CREATE PROCEDURE check_salary(IN emp_id INT)
BEGIN
    DECLARE emp_sal DECIMAL(10,2);
    SELECT salary INTO emp_sal FROM employees
    WHERE id = emp_id;
    IF emp_sal > 70000 THEN
        SELECT 'High Salary' AS result;
    ELSE
        SELECT 'Normal Salary' AS result;
    END IF;
END //
DELIMITER ;
```

12.9 Advantages of Stored Procedures

Feature	Description
Reusability	Can be executed multiple times
Better Performance	Reduces network traffic
Security	Hides complex logic
Maintainability	Centralized business logic
Support for Complex Logic	IF, LOOP, variables, etc.
Data Integrity	Ensures consistent operations

12.10 Common Use Cases

- Complex business logic
- Data validation
- Batch processing
- Periodic tasks (with events)
- Generating reports
- Reusable database operations
- Encapsulating sensitive queries

12.11 Best Practices

- Use meaningful names
- Keep procedures modular
- Use parameters for flexibility
- Handle exceptions properly
- Avoid very long procedures
- Comment your code
- Use transactions when needed
- Test before using in production

Rahul Yadav

13. Triggers in MySQL

13.1 What is a Trigger ?

- A trigger is a special type of stored program that automatically executes when a specified event occurs on a table.
- It runs automatically, you do not call it manually.
- Triggers are mainly used for auditing, logging, data validation and maintaining business rules.
- Types: BEFORE and AFTER triggers.
- Events: INSERT, UPDATE, DELETE.

13.2 How Triggers Work ?

DML Operation (INSERT / UPDATE / DELETE) on a table → Trigger (Automatically Executed) → Perform custom logic (e.g. audit log, validation, etc.)

Key Points:

- Trigger is associated with a table.
- Executes automatically when a DML event occurs.
- Can access NEW (new values) and OLD (old values).
- Can be used to enforce business rules and maintain audit logs.
- Cannot be called manually like a stored procedure.

13.3 Types of Triggers

Type	Description	When it Executes
BEFORE INSERT	Executes before a new row is inserted	Before INSERT
AFTER INSERT	Executes after a new row is inserted	After INSERT
BEFORE UPDATE	Executes before a row is updated	Before UPDATE
AFTER UPDATE	Executes after a row is updated	After UPDATE
BEFORE DELETE	Executes before a row is deleted	Before DELETE
AFTER DELETE	Executes after a row is deleted	After DELETE

13.4 NEW and OLD Keywords

Keyword	Used In	Description
NEW.column	INSERT, UPDATE	Refers to the new value of the column
OLD.column	UPDATE, DELETE	Refers to the old value of the column

- In INSERT → NEW is available, OLD is not available.
- In DELETE → OLD is available, NEW is not available.
- In UPDATE → Both NEW and OLD are available.

13.5 Simple Trigger Example (AFTER INSERT)

Log every new employee insertion into an audit table.

```
DELIMITER //
CREATE TRIGGER trg_emp_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit
    (emp_id, action, action_time)
    VALUES (NEW.id, 'INSERT', NOW());
END //
DELIMITER ;
```

Tables

employees (id, name, salary)

employee_audit (emp_id, action, action_time)

13.6 Trigger Example (BEFORE INSERT) – Data Validation

Prevent inserting negative salary.

```
DELIMITER //
CREATE TRIGGER trg_check_salary
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END //
DELIMITER ;
```

Result

If we try to insert a negative salary, the trigger will throw an error and the record will not be inserted.

13.7 Trigger Example (AFTER UPDATE)

Log salary changes in audit table.

```
DELIMITER //
CREATE TRIGGER trg_emp_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO salary_audit
        (emp_id, old_salary, new_salary, changed_on)
        VALUES (NEW.id, OLD.salary, NEW.salary, NOW());
    END IF;
END //
DELIMITER ;
```

13.8 Trigger Example (BEFORE DELETE)

Prevent deleting important records.

```
DELIMITER //
CREATE TRIGGER trg_prevent_delete
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
    IF OLD.department = 'IT' THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cannot delete IT department employees';
    END IF;
END //
DELIMITER ;
```

13.9 Real World Use Cases

- Audit logging (track INSERT, UPDATE, DELETE).
- Data validation (e.g. prevent invalid data).
- Maintain history of changed data.
- Automatically update related tables.
- Enforce business rules.
- Track critical changes (e.g. salary, status).
- Maintain summary or aggregate tables.

13.10 Advantages and Disadvantages

Advantages	Disadvantages
• Automatic execution	• Can impact performance
• Maintain data integrity	• Complex to debug
• Useful for auditing and logging	• Can make database logic harder to maintain
• Enforce business rules	• Not suitable for very complex logic
• Reduce application side logic	• Overuse can lead to maintenance issues

13.11 Best Practices

- Use meaningful trigger names.
- Keep trigger logic simple.
- Avoid long-running operations.
- Use triggers only when necessary.
- Handle exceptions properly.
- Test triggers in non-production environment.
- Document the purpose of each trigger.

Rahul Yadav

14. Transactions in MySQL

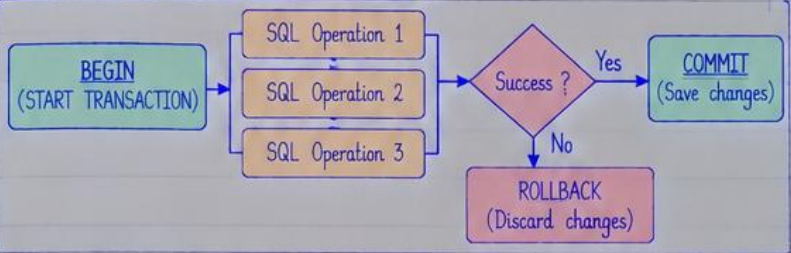
14.1 What is a Transaction ?

- A transaction is a sequence of SQL operations that are executed as a single unit of work.
- Either all the operations are committed (success) or all are rolled back (failure).
- It ensures data consistency and integrity.
- Useful when multiple related operations must succeed or fail together.
- MySQL uses ACID properties to maintain reliable transactions.

14.2 ACID Properties

Property	Description
Atomicity	All operations succeed or none are applied.
Consistency	Database remains in a valid state after transaction.
Isolation	Transactions are isolated from each other.
Durability	Once committed, data is permanently saved even if system fails.

14.3 How Transactions Work ?



14.4 Basic Transaction Syntax

```
START TRANSACTION;  -- or BEGIN;
-- SQL statements
COMMIT;  -- to save changes
-- OR
ROLLBACK;  -- to undo changes
```

14.5 Simple Transaction Example

Transfer money between two accounts (Bank example).

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 5000
WHERE account_id = 1;

UPDATE accounts SET balance = balance + 5000
WHERE account_id = 2;

COMMIT;  -- if both queries succeed
-- ROLLBACK;  -- if any query fails
```

Before Transaction

account_id	name	balance
1	Rahul	50000
2	Amit	30000

Transfer 5000 from 1 to 2

After Transaction

account_id	name	balance
1	Rahul	45000
2	Amit	35000

14.6 Transaction with Error Handling

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 5000 WHERE account_id = 1;
-- Suppose next query fails (invalid account_id)
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 99;

-- If error occurs, rollback
ROLLBACK;
```

14.7 SAVEPOINT in Transaction

- Used to create a point within a transaction.
- Allows partial rollback to a specific point.
- Useful in complex transactions.

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 5000 WHERE account_id = 1;
SAVEPOINT sp1;

UPDATE accounts SET balance = balance + 5000 WHERE account_id = 2;
-- If something goes wrong, rollback to savepoint
ROLLBACK TO sp1;
-- or commit all changes
COMMIT;
```

14.8 Transaction Isolation Levels

- Defines how multiple transactions see each other's data.
- Helps to avoid issues like dirty reads, non-repeatable reads and phantom reads.
- MySQL supports 4 isolation levels.

Isolation Level	Description
READ UNCOMMITTED	Can see uncommitted changes (dirty reads).
READ COMMITTED	Can see only committed changes.
REPEATABLE READ	Default level in MySQL. Prevents non-repeatable reads.
SERIALIZABLE	Highest isolation, prevents phantom reads. But slower performance.

14.9 Common Problems without Transactions

Problem	Description
Data Inconsistency	Partial data updates if one query fails.
Dirty Read	Reading uncommitted data from another transaction.
Non-Repeatable Read	Same query returns different results.
Phantom Read	New rows appear in repeated queries.
Lost Update	One transaction overwrites another transaction's changes.

14.10 Best Practices

- Use transactions for related operations.
- Keep transactions short and simple.
- Use appropriate isolation level.
- Handle errors using ROLLBACK.
- Use SAVEPOINT for complex scenarios.
- Avoid long-running transactions.
- Test transaction logic before production.

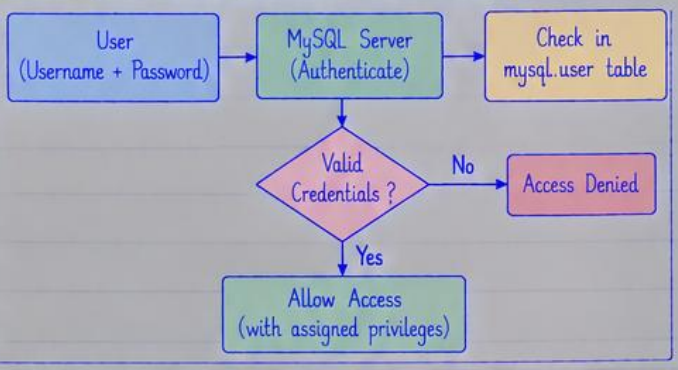
Rahul Yadav

15. User Management and Privileges in MySQL

15.1 What is User Management ?

- User management in MySQL is used to create, manage and control database users.
- Each user can have specific privileges to access databases, tables and perform operations.
- Helps in security, access control and maintaining data integrity.
- We can grant or revoke privileges based on roles and requirements.

15.2 How User Authentication Works ?



15.3 Create a New User

```
CREATE USER 'rahul'@'localhost'
  IDENTIFIED BY 'Rahul@123';
```

Notes:

- 'rahul' → username
- 'localhost' → host (can also use '%' for any host)
- 'Rahul@123' → password

15.4 Create User for Any Host

```
CREATE USER 'app_user'@'%'
 IDENTIFIED BY 'App@123';
```

Note:

- % allows the user to connect from any host.

15.5 View Existing Users

```
SELECT user, host
FROM mysql.user;
```

Output (example)

user	Host
root	localhost
rahul	localhost
app_user	%

15.6 Grant Privileges

Grant specific privileges to a user.

```
GRANT SELECT, INSERT, UPDATE
ON bank_db.*
TO 'rahul'@'localhost';
```

Note:

- Multiple privileges can be granted.
- bank_db.* means all tables in bank_db database.

15.7 Common Privileges in MySQL

Privilege	Description
SELECT	Read data from tables
INSERT	Insert new data
UPDATE	Modify existing data
DELETE	Delete data
CREATE	Create tables or databases
DROP	Drop tables or databases
ALTER	Modify table structure
INDEX	Create or drop indexes
ALL PRIVILEGES	Grant all privileges

15.8 Grant All Privileges

```
GRANT ALL PRIVILEGES
ON bank_db.*
TO 'admin'@'localhost';
```

Grant with GRANT OPTION

```
GRANT ALL PRIVILEGES
ON bank_db.*
TO 'admin'@'localhost'
WITH GRANT OPTION;
```

WITH GRANT OPTION allows the user to grant privileges to other users.

15.9 Flush Privileges

- After creating, granting or revoking privileges, run FLUSH PRIVILEGES.
- This reloads the grant tables.

```
FLUSH PRIVILEGES;
```

15.10 Revoke Privileges

Remove specific privileges from a user.

```
REVOKE INSERT, UPDATE
ON bank_db.*
FROM 'rahul'@'localhost';
```

15.11 Drop a User

Remove a user from MySQL.

```
DROP USER 'rahul'@'localhost';
```

15.12 Manage User Password

Change password using ALTER USER (Recommended)

```
ALTER USER 'rahul'@'localhost'
  IDENTIFIED BY 'New@123';
```

Other way (older syntax)

```
SET PASSWORD FOR 'rahul'@'localhost' =
  PASSWORD('New@123');
```

15.13 User Host Examples

Host Value	Meaning
localhost	Only local server
%	Any host (any IP)
192.168.1.%	Any IP in this range
10.0.0.5	Specific IP address
example.com	Specific domain

15.14 Check User Privileges

Show privileges for current user.

```
SHOW GRANTS;
```

Show privileges for a specific user.

```
SHOW GRANTS FOR 'rahul'@'localhost';
```

15.15 Best Practices

- Create separate users for applications.
- Grant only required privileges.
- Avoid using root user in applications.
- Use strong passwords.
- Regularly review and revoke unused users.
- Use host restrictions instead of '%' when possible.
- Keep user privileges as minimal as required.

15.16 Real World Use Cases

- Create read-only users for reporting.
- Create separate users for development, staging and production.
- Restrict users to specific IP ranges for security.
- Use different users for different microservices.
- Revoke access when an employee leaves the organization.
- Maintain audit logs for user activities.
- Implement the principle of least privilege.

Rahul Yadav

16. Backup and Recovery in MySQL

16.1 What is Backup and Recovery ?

- Backup is the process of creating a copy of your database so that it can be restored in case of data loss.
- Recovery is the process of restoring the database from a backup.
- Helps to protect data from accidental deletion, hardware failure, corruption, etc.
- MySQL provides multiple ways to take backup and recover data.

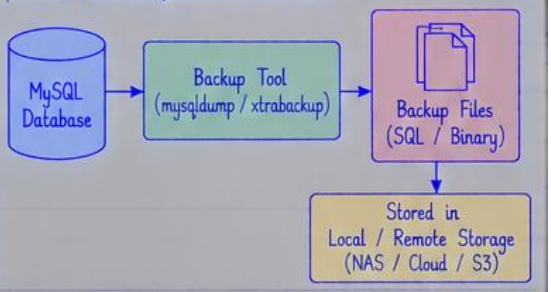
16.2 Why Backup is Important ?

- Protects against data loss.
- Helps in disaster recovery.
- Useful for migration and version upgrades.
- Can be used to recover from human errors (accidental DELETE/UPDATE).
- Ensures business continuity.
- Required for compliance and auditing.

16.3 Types of Backup

Type	Description	Pros	Cons
Full Backup	Complete copy of the database.	Simple to restore.	Large size, more time.
Incremental Backup	Only changes since last backup.	Smaller size, faster.	Slower restore process.
Differential Backup	Changes since last full backup.	Faster restore than incremental.	Larger size than incremental.

16.4 Backup Architecture



16.5 Backup using mysqldump (Logical Backup)

```
# Backup single database
mysqldump -u root -p bank_db > bank_db_backup.sql

# Backup multiple databases
mysqldump -u root -p --databases db1 db2 > multiple_db_backup.sql

# Backup all databases
mysqldump -u root -p --all-databases > all_databases_backup.sql

# Backup with compression
mysqldump -u root -p bank_db | gzip > bank_db_backup.sql.gz
```

16.6 Backup using MySQL Workbench

- Open MySQL Workbench.
- Go to Server → Data Export.
- Select database(s) to export.
- Choose export location.
- Click "Start Export".
- It creates a .sql file which can be used for recovery.

16.7 Backup using xtrabackup (Physical Backup)

```
# Full backup
xtrabackup --backup --target-dir=/backup/full
  --user=root --password=your_password

# Prepare backup
xtrabackup --prepare --target-dir=/backup/full

# Copy to another location
xtrabackup --copy-back --target-dir=/backup/full
```

16.8 Restore using mysqldump Backup

```
# Restore single database
mysql -u root -p bank_db < bank_db_backup.sql

# Restore all databases
mysql -u root -p < all_databases_backup.sql

# Restore from compressed file
gunzip < bank_db_backup.sql.gz | mysql -u root -p bank_db
```

16.9 Point-in-Time Recovery (Binary Logs)

- Binary logs record all changes (INSERT, UPDATE, DELETE).
- Can be used to recover to a specific point in time.
- Enable binary logging in my.cnf:

```
[mysqld]
log-bin=mysql-bin
binlog-format=ROW
expire_logs_days=7
```

Restore to a specific time:

```
mysqlbinlog --stop-datetime=
"2024-01-01 12:00:00"
mysql-bin.000001 |
mysql -u root -p bank_db
```

16.10 Backup Storage Best Practices

- Store backups in multiple locations.
- Keep at least one copy in a different server or cloud (e.g., AWS S3).
- Use compression to save space.
- Regularly verify backup files.
- Automate backups using cron jobs.
- Keep retention policy (e.g., 7 days, 30 days).
- Encrypt backup files for security.

16.11 Common Issues and Solutions

Issue	Cause	Solution
Backup file too large	Large database	Use compression or incremental backup
Restore fails	Version mismatch	Use compatible MySQL version
Missing data after restore	Incomplete backup	Check backup logs and retry
Corrupted backup file	Disk/storage issue	Verify file integrity (checksum)
Permission denied	Insufficient privileges	Use proper user and grant privileges

16.12 Best Practices & Real World Use Cases

- Automate daily backups using cron or scheduler.
- Keep both logical and physical backups.
- Test restore process regularly.
- Use binary logs for point-in-time recovery.
- Monitor backup status and storage space.
- Use cloud storage for disaster recovery.
- Maintain backup documentation.
- Common use cases: disaster recovery, migration, version upgrade, accidental delete recovery.

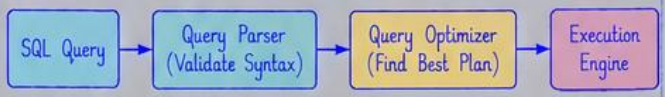
Rahul Yadav

17. Performance Optimization in MySQL

17.1 What is Query Optimization ?

- Query optimization is the process of making SQL queries run faster and use fewer resources.
- Helps to improve application performance.
- Reduces response time and server load.
- MySQL uses a query optimizer to find the best execution plan.
- We can optimize queries using indexing, proper query writing, configuration tuning, etc.

17.2 How MySQL Query Optimization Works ?



Optimizer considers:

- Table statistics
- Indexes available
- Query structure (JOIN, WHERE, ORDER BY, GROUP BY)
- Estimated cost of different execution plans
- Chooses the most efficient execution plan

17.3 Use EXPLAIN to Analyze Query

EXPLAIN shows how MySQL will execute the query.

```
EXPLAIN SELECT id, name, department
        FROM employees
        WHERE department = 'IT';
```

Output (example)

id	select_type	table	type	rows	Extra
1	SIMPLE	employees	ref	10	Using index condition

Important Columns

type	Type of join (performance indicator)
rows	Estimated number of rows to scan
Extra	Additional information (e.g., Using index, Using temporary)

17.4 Indexing for Better Performance

- Indexes help to find data quickly without scanning the entire table.
- Use indexes on columns used in WHERE, JOIN, ORDER BY, GROUP BY.
- Avoid too many indexes (slows down writes).

Create an Index

```
CREATE INDEX idx_department
ON employees(department);
```

Composite Index (Multiple Columns)

```
CREATE INDEX idx_dept_salary
ON employees(department, salary);
```

17.5 Types of Indexes

Index Type	Description
Primary Index	Created automatically on PRIMARY KEY
Unique Index	Ensures all values are unique
Normal Index	Used for faster searching
Composite Index	Index on multiple columns
Full Text Index	Used for text search (LIKE, MATCH)
Spatial Index	Used for spatial data types

17.6 Indexing Example

Without Index (Full Table Scan)

```
EXPLAIN SELECT * FROM employees
        WHERE department = 'IT';
  type: ALL      (scans entire table)
  rows: 50000
```

With Index

```
CREATE INDEX idx_dept
ON employees(department);
```

```
EXPLAIN SELECT * FROM employees
        WHERE department = 'IT';
  type: ref      (uses index)
  rows: 10
```

17.7 Optimize JOIN Queries

- Use proper indexes on join columns.
- Avoid unnecessary columns in SELECT.
- Use INNER JOIN instead of LEFT JOIN when possible.

Example:

```
SELECT e.id, e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.id
WHERE d.department_name = 'IT';
```

17.8 Avoid SELECT *

- Select only required columns.
- It reduces I/O and network overhead.

Bad Query

```
SELECT * FROM employees;
```

Optimized Query

```
SELECT id, name, department, salary
FROM employees;
```

17.9 Optimize WHERE Clause

- Use indexes on filtered columns.
- Avoid functions on indexed columns (e.g., DATE(column)).
- Use proper data types.

Bad Query (Function on column)

```
SELECT * FROM employees
WHERE YEAR(join_date) = 2024;
```

Optimized Query

```
SELECT * FROM employees
WHERE join_date >= '2024-01-01'
      AND join_date < '2025-01-01';
```

17.10 Optimize ORDER BY and GROUP BY

- Use indexes on columns used in ORDER BY / GROUP BY.
- Avoid sorting large result sets.
- Use LIMIT for pagination.

Bad Query

```
SELECT * FROM employees
ORDER BY salary DESC;
```

Optimized Query

```
CREATE INDEX idx_salary
ON employees(salary);
SELECT id, name, salary
FROM employees
ORDER BY salary DESC
LIMIT 100;
```

17.11 MySQL Configuration Tuning

Parameter	Purpose
innodb_buffer_pool_size	Cache data and indexes (increase for better performance)
query_cache_size	Cache query results (removed in MySQL 8.0)
max_connections	Maximum client connections
sort_buffer_size	Memory for sorting operations
join_buffer_size	Memory for join operations
tmp_table_size	Size of temporary tables

17.12 Best Practices

- Use indexes wisely.
- Write efficient queries.
- Avoid SELECT *.
- Use proper data types.
- Analyze queries using EXPLAIN.
- Keep statistics updated (ANALYZE TABLE).
- Use query caching (if supported).
- Monitor slow queries using slow query log.
- Regularly optimize and maintain tables.

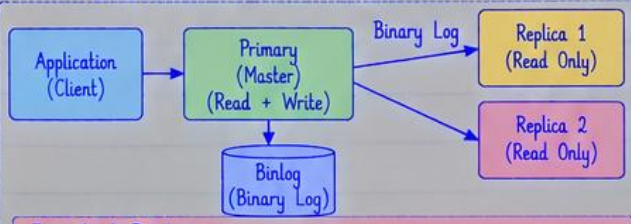
Rahul Yadav

18. Replication and High Availability in MySQL

18.1 What is Replication ?

- Replication is the process of copying data from one MySQL server (Master/Primary) to one or more servers (Replica/Secondary).
- Used for high availability, read scaling and disaster recovery.
- Data changes (INSERT, UPDATE, DELETE) on primary are replicatea to replicas.
- Replication in MySQL is asynchronous by default (can be semi-synchronous).

18.2 Replication Architecture



Important Points:

- Primary server records all changes in Binary Log (binlog).
- Replica reads binlog and applies changes (via I/O and SQL threads).
- Replicas are usually read-only.
- Helps in load balancing and data redundancy.

18.3 Types of Replication

Type	Description	Use Case
Asynchronous	Default replication mode. Replica may have some delay.	Most common, better performance.
Semi-synchronous	Primary waits for at least one replica to acknowledge.	More data safety.
Synchronous	Primary waits for all replicas to apply changes.	High consistency (but slower).

18.4 Replication Components

Component	Description
Binary Log (binlog)	Records all data changes on primary.
I/O Thread (Replica)	Reads binlog from primary.
SQL Thread (Replica)	Applies changes to replica database.
Replication User	User with REPLICATION privileges.
Server ID	Unique id for each MySQL server.

18.5 Configure Primary (Master)

1. Enable binary logging in my.cnf

```
server-id = 1
log_bin = mysql-bin
binlog_format = ROW
expire_logs_days = 7
```

2. Create a replication user

```
CREATE USER 'repl'@'%' IDENTIFIED BY 'repl123';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';
FLUSH PRIVILEGES;
```

3. Check master status

```
SHOW MASTER STATUS;
```

Output (example)

File	Position	Binlog_Do_DB
mysql-bin.000001	154	

18.6 Configure Replica (Slave)

1. Set server-id in my.cnf

```
server-id = 2
read_only = 1
```

2. Configure replication

```
CHANGE REPLICATION SOURCE TO
  SOURCE_HOST = '192.168.1.10',
  SOURCE_USER = 'repl',
  SOURCE_PASSWORD = 'repl123',
  SOURCE_LOG_FILE = 'mysql-bin.000001',
  SOURCE_LOG_POS = 154;
```

3. Start replication

```
START REPLICA;  -- (For MySQL 8.0)
-- START SLAVE;  -- (For MySQL 5.7)
```

4. Check status

```
SHOW REPLICA STATUS\G;    -- (MySQL 8.0)
-- SHOW SLAVE STATUS\G;   -- (MySQL 5.7)
```

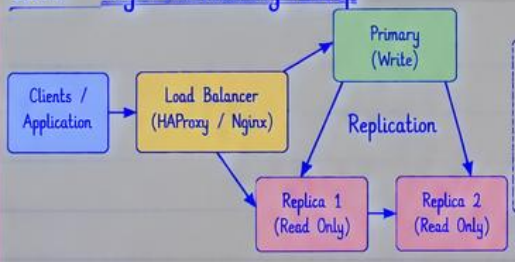
18.7 Replication Status (Important Fields)

Field	Description
Replica_IO_Running	Shows if I/O thread is running (Yes/No)
Replica_SQL_Running	Shows if SQL thread is running (Yes/No)
Seconds_Behind_Source	Time lag in seconds (0 is ideal)
Last_Error	Shows last error if replication fails

18.8 Common Issues and Solutions

Issue	Cause	Solution
Replica not starting	Wrong credentials or server-id	Check user, password and server-id
Replication lag	High load on replica	Optimize queries, increase resources
Duplicate server-id	Same server-id on multiple hosts	Use unique server-id
Table not replicating	Database/table filter	Check replicate-do-db / replicate-ignore-db settings
Replication stopped	Network issue	Check network and restart replica

18.9 High Availability Setup



Benefits:

- High availability
- Read scaling
- Failover support
- Disaster recovery
- Better performance

18.10 Best Practices

- Use replication for read-heavy workloads.
- Monitor replication lag regularly.
- Use semi-synchronous replication for critical data.
- Have proper backup along with replication.
- Use unique server-id for each server.
- Monitor using monitoring tools (e.g., Prometheus, Percona).
- Test failover process regularly.
- Keep binlog retention for recovery.
- Use appropriate hardware and network.

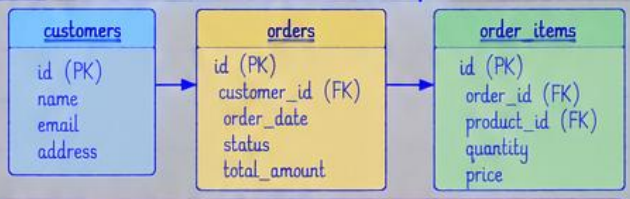
Rahul Yadav

19. Real World Use Cases and Best Practices in MySQL

19.1 Real World Use Cases of MySQL

- Banking and Finance (transactions, customer data)
- E-commerce (products, orders, inventory)
- Social Media (users, posts, relationships)
- Healthcare (patient records, appointments)
- Telecom (customer data, call records)
- Logistics (tracking, shipment data)
- Analytics (reporting, data warehouse)
- Internal Tools (HR, employee management)
- Microservices (as a persistent data store).

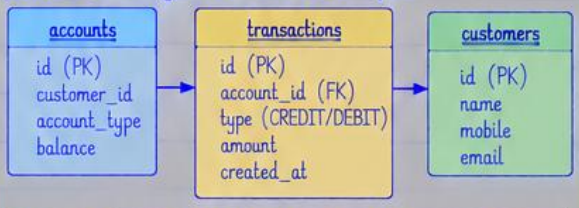
19.2 E-commerce Use Case Example



Example Query : Get total amount per customer

```
SELECT c.id, c.name, SUM(o.total_amount) as total_spent
FROM customers c
JOIN orders o ON c.id = o.customer_id
GROUP BY c.id, c.name;
```

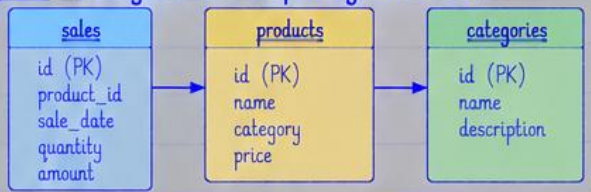
19.3 Banking Use Case Example



Example Query : Get last 5 transactions for an account

```
SELECT id, type, amount, created_at
FROM transactions
WHERE account_id = 101
ORDER BY created_at DESC
LIMIT 5;
```

19.4 Analytics and Reporting Use Case



Example Query : Monthly sales report by category

```
SELECT c.name as category,
       DATE_FORMAT(s.sale_date, '%Y-%m') as month,
       SUM(s.amount) as total_sales
FROM sales s
JOIN products p ON s.product_id = p.id
JOIN categories c ON p.category_id = c.id
GROUP BY c.name, DATE_FORMAT(s.sale_date, '%Y-%m')
ORDER BY month, total_sales DESC;
```

19.5 Best Practices for Designing Tables

1. Use proper data types (INT, VARCHAR, DATE, etc).
2. Define PRIMARY KEY for each table.
3. Use appropriate indexes for frequently searched columns.
4. Normalize tables to reduce data duplication.
5. Use foreign keys to maintain referential integrity.
6. Set DEFAULT values where applicable.
7. Avoid using NULL if possible.
8. Use meaningful and consistent naming conventions.
9. Use timestamps for audit (created_at, updated_at).
10. Consider partitioning for very large tables.

19.6 Best Practices for Writing Queries

1. Use indexes on WHERE, JOIN and ORDER BY columns.
2. Select only required columns (avoid SELECT *).
3. Use EXPLAIN to analyze query performance.
4. Avoid functions on indexed columns (e.g., DATE(column)).
5. Use appropriate JOIN type (INNER, LEFT, etc).
6. Use pagination (LIMIT) for large result sets.
7. Use prepared statements to prevent SQL injection.
8. Optimize subqueries and use proper joins instead.
9. Filter data as early as possible.
10. Keep queries simple and readable.

19.7 Security Best Practices

1. Use strong passwords for MySQL users.
2. Follow least privilege principle (grant only required permissions).
3. Restrict access using host (not '%').
4. Use SSL/TLS for remote connections.
5. Regularly review user accounts and privileges.
6. Keep MySQL version updated.
7. Enable audit logs for sensitive operations.
8. Use firewall and network restrictions.
9. Backup user accounts and configurations.
10. Monitor for unusual activity (failed logins, etc).

19.8 Maintenance Best Practices

1. Regular backups (full + incremental).
2. Monitor disk space and table size.
3. Optimize tables using OPTIMIZE TABLE.
4. Regularly check and rebuild indexes if needed.
5. Monitor slow queries using slow query log.
6. Clean up old or unused data (archiving).
7. Keep an eye on replication lag (if using replication).
8. Update statistics for better query plans.
9. Use monitoring tools (e.g., Prometheus, Grafana, Percona).
10. Plan for capacity and scaling (vertical or horizontal).

19.9 Common Mistakes to Avoid

Mistake	Impact
Using SELECT *	Unnecessary data, slow queries
No index on search columns	Full table scan, poor performance
Storing large files in DB	Increases DB size, slow performance
Not using transactions	Data inconsistency
Ignoring backup	Risk of data loss
Using weak passwords	Security risk
Not monitoring DB	Issues go unnoticed

19.10 Real World Tips for Production

Tip	Description
Use Master-Replica setup	Improves read performance and availability
Use connection pooling	Handle large number of connections efficiently
Enable query cache (if supported)	Cache frequent queries (MySQL < 8.0)
Use partitioning	Better performance for large tables
Archiving old data	Keep active data small and fast
Set up monitoring and alerts	Monitor CPU, memory, disk, slow queries
Plan for disaster recovery	Have tested backup and restore process
Use automation (scripts/tools)	Regular maintenance and backups

Rahul Yadav